



Runtime Verification in Real-Time with the Copilot Language: A Tutorial

Ivan Perez¹(✉), Alwyn E. Goodloe², and Frank Dedden³

¹ KBR @ NASA Ames Research Center, Houston, USA
`ivan.perezdominguez@nasa.gov`

² NASA Langley Research Center, Hampton, USA
`a.goodloe@nasa.gov`

³ System F Computing, Rotterdam, The Netherlands
`frank@systemf.dev`

Abstract. Ultra-critical systems require high-level assurance, which cannot always be guaranteed at compile time. The use of runtime verification (RV) enables monitoring of these systems during runtime, to detect illegal states early and limit their potential consequences. This paper is a tutorial on RV using Copilot, an open-source runtime verification framework actively used by NASA to carry out experiments with robots and unmanned aerial vehicles. Copilot monitors are written in a compositional, stream-based language, which the framework automatically translates into real-time C code that satisfies static memory requirements suitable to run on embedded hardware. Copilot includes multiple libraries that extend the core functionality with higher-level constructs, Boyer-Moore majority voting, and a variety of Temporal Logics (TL), resulting in robust, high-level specifications that are easier to understand than their traditional counterparts.

1 Introduction

Embedded systems are used in a wide range of applications, ranging from televisions and cellphones to automobiles, aircraft and ships. In all of these applications, we want the system to function correctly, but those systems that are *safety critical*, where failure can result in injury or death of a human, warrant special attention [14]. To achieve the necessary level of reliability, both hardware and software of safety-critical systems need to be of very high quality. Formal verification techniques are one method for achieving the level of reliability required in safety-critical systems. Generally, formal verification is based on mathematically proving correctness properties of a model of the system under study. Although there have been considerable advances in industrial-scale formal methods, they remain too expensive to apply them on most projects.

Runtime verification (RV) [2, 10, 11] is a verification technique that has the potential to enable the safe operation of safety-critical systems that are too complex to formally verify or fully test. In RV, the system is monitored *during execution*, to detect and respond to property violations that take place during operation. When an unsafe state is detected, the monitor invokes system-specific

routines to report or recover from the violation. RV detects when properties are violated during *during runtime*, so it does not constitute a proof of correctness, but is a significant improvement over testing alone.

RV concerns itself with the detection of faults and off-nominal conditions. Upon detection of a property violation, the system under observation (SUO) could switch to a backup system, transfer control to a pilot or operator on the ground, degrade performance, log the error for posterior analysis, or take some other corrective action. The specifics of how faults are handled are mission-specific, and not the subject of RV itself.

Correct implementation of these monitors and the RV subsystem is crucial for the safe operation of the complete system and the success of the mission as a whole. The introduction of errors in the RV subsystem could disable or affect other subsystems, or lead to suboptimal deviations from a mission. In resource-constrained environments and time-critical systems, runtime monitors are commonly implemented in C due to performance and memory constraints. This results in low-level code that is error-prone, hard to understand and difficult to maintain.

In this paper we present Copilot, a runtime verification framework to write high-level specifications. Copilot is implemented as a stream-based, deeply embedded domain-specific language in Haskell. Streams are used to specify monitors, which denote functions that detect when properties are violated. Once a monitor is triggered, a user-defined function is called to take appropriate action. Our framework provides a constrained set of operations to define and combine streams, which guarantees that they are well-formed. The language also relies on dependent types, to enable safe use of non-primitive data structures, like structs and arrays. Copilot translates definitions into a MISRA-compliant subset of C99. MISRA C is a set of guidelines for developing C code targeting real-time embedded systems, which promotes code safety and security. For instance, the guidelines constrain the design with predictable memory requirements and real-time guarantees. To run the monitor on an embedded system, the generated C99 code can be compiled for a target platform and integrated with the system under observation. Additional Copilot libraries extend the core language with higher-level constructs, and temporal logic [15,20].

This paper is structured as follows: Sect. 2 provides a brief history of the Copilot framework. Section 3 is an introduction to the trace theoretic view of RV, sampling, and RV instrumentation. Section 4 has a “hello world” style motivating example. Section 5 introduces the Copilot specification language which simplifies prior versions of the language [17,19] and extends it with notions of arrays and structs. Section 6 demonstrates how to specify runtime monitors using basic stream-level functions as well as different temporal logics. In Sect. 7, we discuss how to integrate Copilot monitors into the larger system being monitored.

2 History

The Copilot project began in September 2008 when NASA awarded Galois, Inc. and the National Institute of Aerospace a contract (“Monitor Synthesis for Software Health Management”¹) to perform research in the area of runtime verification applied to hard real-time distributed systems with a concentration on avionics. Focusing on hard real-time embedded systems imposed the following constraints: monitors should not affect the system under observation (SUO) in a way that changes the functionality of the system, requires re-certification, interferes with timing, or exceeds size, weight, and power (SWAP) constraints. The four most significant design decisions made in the early days were: that the RV framework that came to be called Copilot would be implemented as a Haskell-embedded domain-specific language (EDSL), that Copilot would favor lightweight instrumentation and thus employ sampling as a means to observe the executing system, that runtime monitors must run in constant space and constant time, and the fourth significant design decision was that the specification language would be a stream-based data-flow language inspired by Lustre [4] and LOLA [6]. Note that Lustre has an existing user base in the aerospace industry, meaning that stream-based languages have proven acceptable in that domain.

The first Copilot prototypes focused on evolving the specification language and constructing an interpreter. During this time period, several open-source tools were used to generate C code from the specification. Copilot was demonstrated in numerous flight tests, including the first demonstration of Byzantine fault-tolerant RV [19]. The architecture of the older Copilot 2.0 framework is described in [18] along with a description of how lightweight formal methods were applied to the problem of monitor correctness. To address the challenge of ensuring that a formal specification is correct, research was conducted integrating model checking and SMT capabilities into the Copilot 2.0 framework [9, 13].

Early incarnations of Copilot were very much research efforts and could be very clumsy to use in practice because, in embedded systems, many variables are stored as either C structs or C arrays, and Copilot could not handle these data structures. Given that we found no existing tool that supported generating code with arrays and structs, it was decided to build a new C code generator from scratch that could accommodate the needs of Copilot. This necessitated a considerable rewrite of the whole Copilot framework.

The decision to rewrite the framework corresponded with more stringent demands from users who wanted Copilot to generate monitors that were trustworthy enough to be certified by NASA as critical flight code. NASA began an initiative focusing on *high-assurance RV* [9]. The project adapted more structured software engineering processes following the NASA standard NPR7150.2C [16] that required extensive documentation such as documenting the architecture and design, software test plans, and coding style guides. In order to satisfy requisite assurance demands, unit tests were constructed for every module using Haskell’s QuickCheck tool, and Galois Inc developed the

¹ NASA award NNL08AA19B, order NNL08AD13T.

Copilot Verifier that generates a mathematical proof that the generated monitor and specification are bisimilar [22]. In addition to the C code generator described in this tutorial, a prototype backend generating BlueSpec² has been developed enabling implementing monitors on FPGAs and exploratory work has begun on a Copilot backend for creating Rust monitors. Copilot was certified as a NASA Software Engineering tool (NPR7150.2, Class D) in June 2023 and Copilot developers continue to work with flight-safety groups at several NASA centers to improve the utility of Copilot for use in monitoring critical flight systems.

Today, Copilot continues to be developed as an open-source project,³ with new releases being published every two months. Our Github repository contains detailed installation instructions for multiple operating systems. Users who wish to investigate how we meet some of the requirements of NPR7150.2 can visit our repository, specifically the issues, the pull requests, and the commit messages, where we leave evidence that can be used to audit our software development process. Users from the community are welcome to participate by providing contributions, asking questions, proposing new features, and by extending and using Copilot.

3 Background

Runtime verification is a dynamic software analysis technique that detects if a formally specified property is violated during an execution of a program or system. Copilot is not only a specification language, but a framework that transforms the specification into an executable monitor along with supporting code needed to observe the executing program. In the remainder of this section, we provide some background material that should aid understanding of RV in general and Copilot in particular.

3.1 Trace Theory

In order to check if an executing system satisfies a specification at runtime, the monitor must be able to observe a trace capturing the evolving state or events during execution. A *trace* [21] provides a view of an executing system that captures the evolving state or events that occur during a single run of a system. Suppose E is the set of states or events of an executing system and E^* is the set of all finite sequences of elements of E , then a trace $\tau \in E^*$ is a finite sequence of observed events or states. In RV, a trace is checked against a formal specification ϕ expressed in a formal logic. A specification denotes the set of traces that satisfy it. An RV monitor then must check that a trace τ is a member of the language of a specification ϕ , formally $\tau \in \mathcal{L}(\phi)$. This is sometimes phrased as “ τ satisfies the specification ϕ ” and expressed as $\tau \models \phi$. The SUO must be instrumented to extract the trace from the executing program and an RV framework should support the instrumentation.

² <https://github.com/B-Lang-org>.

³ <https://github.com/copilot-language/copilot>.

3.2 Sampling

The idea of sampling representative data from a large set of data is well established in engineering. For instance, in digital signal processing, a signal such as music is sampled at a very high rate to obtain enough discrete points to represent the physical sound wave. The fidelity of the recording is dependent on the sampling rate.

Monitoring based on sampling state variables has historically been disregarded as a runtime monitoring approach, for good reason: without the assumption of synchrony between the monitor and observed system, monitoring via sampling may lead to false positives and false negatives [7]. For example, consider the property $(0, 1, 1)^*$, written as a regular expression, denoting the sequence of values a monitored variable may take (that is, in nominal conditions, we would expect the monitored variable to take the sequence of values of the shape $0, 1, 1, 0, 1, 1, 0, 1, 1, \dots$). Depending on the specific times when the RV system samples the variable, both false negatives (the monitor erroneously rejects the sequence of values) and false positives (the monitor erroneously accepts the sequence) are possible. For example, if the actual sequence of values is $0, 1, 1, 0, 1, 1$, then an observation of $0, 1, 1, 1, 1$ will lead to a false negative, because a value has been skipped (Fig. 1). If the actual sequence is $0, 1, 0, 1$, then an observation of $0, 1, 1, 0, 1, 1$ will lead to a false positive, because a value is sampled twice (Fig. 2).

However, in a hard real-time context, sampling is a suitable strategy. Often, the purpose of real-time programs is to deliver output signals at a predictable rate. Under the assumption that the monitor and the observed program share a global clock and a static periodic schedule, false positives are possible, but false negatives are not. Moreover, in the context of cyber-physical systems, the data comes from sensors measuring physical attributes such as GPS coordinates, air speed, and actuation signals. Such continuous signals do not change abruptly and hence sampling suffices as long as it is performed at a suitable rate.

Many RV frameworks utilize inline monitors in the observed program to avoid the aforementioned problems with sampling. However, inlining monitors changes the real-time behavior of the observed program, perhaps in unpredictable ways. Introducing such unpredictability is not a viable solution for ultra-critical hard real-time systems. With Copilot’s sampling-based approach, the monitor can be integrated as a separate scheduled process during available time slices (this is made possible by generating efficient constant-time monitors). Indeed, Copilot monitors may even be scheduled on a separate processor (albeit doing so requires additional synchronization mechanisms), ensuring time and space partitioning from the observed programs. Other RV frameworks targeting cyber-physical systems, such as R2U2 [12], have made the same design decision.

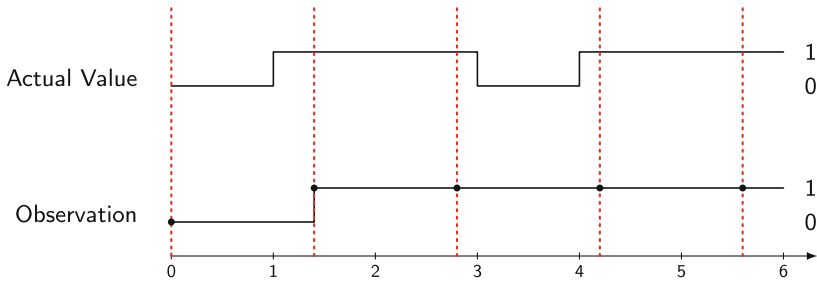


Fig. 1. Diagram representing values taken by a variable at regular points in time, and the observations taking place (at regular, but different, times), with a dotted line indicating the instants when the observations are taken. In this example, observations are made at a slower rate than values change, which may happen in a realistic scenario. In this case, the observation will lead to a false negative result compared to the actual value (the actual value is 0, 1, 1, 0, 1, 1, which conforms to the regular expression we are recognizing, whereas the observation is 0, 1, 1, 1, 1, 1, which does not).

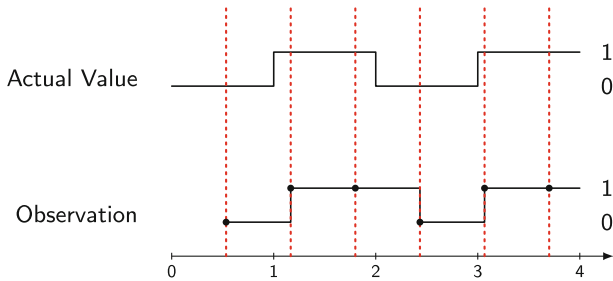


Fig. 2. Diagram representing values taken by a variable at regular points in time, and the observations taking place (at regular, but different, times), with a dotted line indicating the instants when the observations are taken. Notice that, in this example, the observation starts late compared to when the actual value is first set, which may happen in a realistic scenario. In this case, the observation renders false positive results compared to the actual value (the actual value is 0, 1, 0, 1, which does not conform to the regular expression we are recognizing, whereas the observation is 0, 1, 1, 0, 1, 1, which does conform to the regular expression).

4 Hello, Copilot!

The purpose of this section is to help the reader gain some intuition about the concepts that Copilot is built on, the different parts that make a monitor, and how the tool generates the code.

A Copilot monitor observes a system, analyzes the data being observed, and, if there are any property violations to report, executes functions that address those violations. Copilot does not determine how to fix the violations: its goal is to detect the problem and produce a notification.

Let us illustrate the main ideas in Copilot with a specific example of a collision avoidance system for a plane. When the plane is in cruise mode and governed by the autopilot, the altitude should not drop below a threshold, since that could indicate that there is a problem with the autopilot, the positioning system, or the sensors.

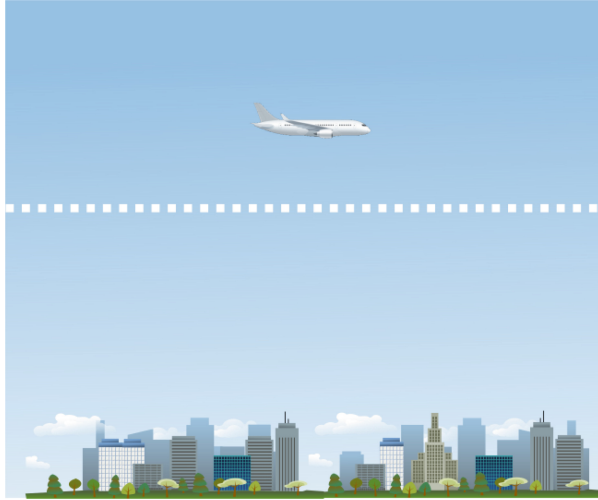


Fig. 3. Illustration of a plane flying over a city, with a dotted line indicating the minimum altitude at which the plane should be flying (not to scale).

When writing a Copilot specification for a monitor, we must consider:

- What properties must be monitored.
- What data is needed i.e. the trace to be captured.
- What functions handle the monitoring violations, and what data is given to them.
- When the monitors are running.

In this particular example, the property that must be monitored is that the altitude of the aircraft is higher than a threshold when the autopilot mode is on. The data needed is 1) knowing whether the autopilot is on, 2) the altitude at any given time, and 3) the minimum altitude. In this example, we'll define input data streams for (1) and (2), and have a configuration parameter for (3) that remains constant throughout the flight. When the Copilot monitor detects that the system has entered an unsafe state, the monitor invokes an error handling function, which in this case will be called **recover**, providing as argument the current altitude and the desired threshold.

A Copilot monitor is an association between a boolean property and a handling function that is executed whenever the property becomes true. Monitors are to be checked at regular intervals or whenever new data is available.

To work with such changing values, Copilot sees data not as static, but as values that change over discrete time. Copilot properties can refer not just to the current values of the data, but also to past values. All data, from inputs to properties to outputs, are seen as streams, or infinite sequences of data samples. A property being monitored is represented by a time-varying boolean, also known as a boolean stream, but streams can carry other kinds of data (numeric, arrays, etc.). The following diagram depicts two streams: a numeric stream `counter`, which starts at zero and increases by one unit at every step, and a boolean stream `evenCounter`, which becomes true when the `counter` is even, and false otherwise.

<code>evenCounter</code>									...
<code>counter</code>	0	1	2	3	4	5	6	7	...
<i>Sample number</i>	1	2	3	4	5	6	7	8	...

At a glance, our monitor looks like this:

```
1 import Language.Copilot
2 import Copilot.Language
3
4 autopilot :: Stream Bool
5 autopilot = extern "autopilot" Nothing
6
7 altitude :: Stream Word64
8 altitude = extern "altitude" Nothing
9
10 threshold :: Stream Word64
11 threshold = constant 10000 -- feet
12
13 property :: Stream Bool
14 property = autopilot ==> (altitude > threshold)
15
16 violation :: Stream Bool
17 violation = not property
18
19 monitor :: Spec
20 monitor = do
21   trigger "recover" violation [ arg altitude, arg threshold ]
```

There are several distinct parts to this specification. Lines 1–2 list libraries that must be imported. Lines 4–8 define two input streams: `autopilot` and

altitude. The former is a stream carrying boolean values, and it is defined externally by an input by the same name. The latter is a stream carrying an unsigned 64-bit integer, and is also defined externally by an input by the same name. Lines 10–11 define an internal stream, also carrying unsigned 64-bit integers, which is constantly 10000.

Lines 13–17 define the desired system property: in this case, if the autopilot is on, the altitude should be greater than the threshold. The property is trivially true if the autopilot is off, as it is defined by an implication. The property that we wish to monitor is violated whenever it becomes false, as defined by **violation**.

Finally, in lines 19–21, the **monitor** is defined by a trigger that associates the handler **recover** to the stream **violation**, with two additional arguments. This means that, whenever violation becomes true, the external handler function **recover** will be called, passing the current values of **altitude** and **threshold** as arguments to the function.

Notice also the text `-- feet` in the definition of **threshold**, which represents a comment we include to help the reader understand the meaning of the constant.

5 The Copilot Specification Language

Copilot is a framework that comprises an RV specification language, and a tool that compiles specifications into C code. Copilot specifications are defined by a series of *triggers*, that is, properties that need to be monitored paired with functions that need to be called when those properties become true. Properties themselves are defined as Boolean-carrying *streams*, using a rich language of stream definitions that includes primitives and combinators, and gives access to external streams defined in C.

5.1 Streams

Streams are infinite successions of values, and constitute the central entity of Copilot specifications. Streams can be defined using primitives, or be built from other streams with a series of combinators. We provide a limited Application Programming Interface (API) to ensure that, by construction, streams are well-formed and can be compiled to efficient C code.

Constant Streams. The simplest **Stream** definition in Copilot is a constant stream of values, for which we provide the primitive **constant** that takes an element and returns a stream consisting of that element at every sample.

Example 1. The stream **true**, which Copilot defines in its Prelude for convenience, represents a constant stream carrying the Boolean value **True** and is defined as:

```
1 true = constant True
```

Because Copilot is a strongly typed domain-specific language, every expression and stream has a unique type. Following the notation of the host language Haskell, the type signature of every top-level definition is normally stated right before, prefaced by `::`. For example, `true`, as defined above, is a stream of Boolean values, which we denote with the type signature `true :: Stream Bool`.

The primitives and combinators that form the Copilot language are also functions that return streams. For example, the primitive `constant` itself is a Haskell function with type:

```
1 constant :: Typed a => a -> Stream a
```

The type signature of this primitive has two parts, separated by the symbol `=>`. On the right-hand side, the expression `a -> Stream a` indicates that `constant` is a function that takes an element of any type, which we call `a`, and returns a `Stream` carrying elements of the same type `a`. On the left-hand side, the expression `Typed a` is a *type constraint* and requires `a` to be an instance of the class `Typed`, denoting types that Copilot knows how to represent in C. We will not cover how to define custom types or instances in this document. Readers interested can consult standard Haskell textbooks to understand classes and instances, and the Copilot API to understand the type class `Typed`.

Including the type signatures explicitly is not always mandatory and it may be convenient to leave them out, especially when building very large and complex expressions. However, Copilot sometimes requires explicit type signatures for expressions that are ambiguous, in order to understand how to generate the corresponding C code. For example, the expression `constant 1` is a valid expression of type `Stream Int32`, but also one of type `Stream Int64` (and several other types). Without some type annotation, Copilot cannot know if it needs to use `uint32_t` or `uint64_t` in C to store the data, and so it requires that we spell out the type of the expression. To minimize the need for type annotations, Copilot provides a family of constant stream-building functions for each primitive type. For example, we can define the constant stream of 1's using 64-bit integer numbers as:

```
1 ones :: Stream Int64
2 ones = constI64 1
```

In this case, the type signature is redundant: because we have used the primitive `constI64`, the compiler knows we mean to build a stream of 64-bit integers. Conversely, we could have instead defined this stream using the ‘constant’ primitive; since we explicitly state the type of the stream, the compiler knows which representation to use. For streams carrying numbers, we can also use the literal number without ‘constant’ or ‘constI64’. The compiler knows that literal numbers just mean constant streams whenever a stream is expected.

Lifting and Point-Wise Function Application. We provide definitions that extend the standard API of each type supported by Copilot to act pointwise on streams. Copilot supports Boolean values (i.e., `Bool`), signed and unsigned

fixed-length integers (i.e., `Word8`, `Word16`, `Word32`, `Word64`, `Int8`, `Int16`, `Int32`, `Int64`), floating point numbers (i.e., `Float`, `Double`), limited structs, and limited arrays. The operators and combinators provided by Copilot are limited to a subset that we can compile to C efficiently. Details on structs and arrays are given in Sects. 5.2 and 5.3 respectively.

Example 2. The standard negation function `not`, operating on Booleans, would normally take one Boolean value and return another Boolean. Copilot defines `not` to operate on *streams of Booleans*. For example, the stream `false`, which holds the constant value `False`, could be defined by applying `not` to negate every value in the `true` stream defined earlier:

```
1 false :: Stream Bool
2 false = not true
```

Streams can contain other values representable in C, like integers and doubles. We overload literal numbers and mathematical operators to work on streams: literals denote constant streams, and operators are applied pointwise. For example, we can define the constant stream carrying the number 4 based on the definition of `ones` from before, as:

```
1 fours :: Stream Int64
2 fours = ones + 3
```

In this definition, the symbol `3` denotes the constant stream of 3's, and the symbol `+` denotes addition of streams carrying numbers, defined pointwise (e.g., the first element of `ones` plus the first element of `3`, the second element of `ones` plus the second element of `3`, and so on).

Temporal Translations. Because streams represent values that change over time, it seems natural to think of how to refer to a past or future value of a stream. Copilot provides two mechanisms to translate a stream in time: delays, which allow us to refer in the present to values in the past, and drops, which allow us to refer in the present to values in the future. Both impose additional constraints to ensure that the resulting specification is well-defined, and that it can be executed in real time.

Delaying a stream requires that we hold the stream's actual value in memory for future use. Unbounded delays (i.e., those in which the amount of elements to hold is potentially unbounded) are known to lead to memory leaks [5]. To make the generated C code efficient and memory usage predictable, we provide very limited ways of delaying streams: streams can be delayed by pre-pending a *fixed number of samples*, with the operator `(++)`, with type:

```
1 (++) :: Typed a => [a] -> Stream a -> Stream a
```

Example 3. We can use the append operator `(++)` to create a stream that is initially `False` and later becomes `True` indefinitely:

```

1 falseThenTrue :: Stream Bool
2 falseThenTrue = [False] ++ true

```

Note that streams can be defined recursively. For example, we can define the stream that alternates between the values `True` and `False` as:

```

1 alternatingStream :: Stream Bool
2 alternatingStream = [True] ++ not alternatingStream

```

Using recursion, like before, we can define a step counter (e.g., $[1, 2, 3, \dots]$) as follows:

```

1 counter :: Stream Int32
2 counter = [1] ++ (counter + 1)

```

Copilot also provides the opposite temporal transformation, dropping elements from a stream, with the function:

```

1 drop :: Typed a => Int -> Stream a -> Stream a

```

Example 4. In the following example, we use `drop` to eliminate the first 2 elements from a stream:

```

1 numbers :: Stream Int64
2 numbers = [1,34,2,9,8,15] ++ numbers
3
4 numbers' :: Stream Int64
5 numbers' = drop 2 numbers    -- 2, 9, 8, 15, 1, 34, 2, 9, ...

```

Dropping elements introduces a potential issue if the elements are not available, which may happen if they come from an external source (e.g., a sensor). This is discussed in the following.

External Streams. To connect Copilot specifications to existing C applications, we provide the primitive `extern` to define a stream based on the value of a global C variable, by indicating its name and its type. Within Copilot, we have no way of guaranteeing that a given variable exists, or that it has the expected type. However, from a specification containing an external stream, Copilot generates a C header file that declares the existence of an extern global variable with a specific type. The use of a variable name that does not exist, or that has the wrong type, would give rise to either an error or a warning when trying to compile and link the generated C code as part of a larger application.

Example 5. Commonly in Copilot specifications, there is a need to access data provided by an external sensor. For example, given a global variable `altitude`, of a type `Word64`, holding the current altitude of the plane, we can define a Copilot specifications as follows:

```

1 altitude :: Stream Word64
2 altitude = extern "altitude" Nothing

```

The additional argument `Nothing` contains an optional list of `Word64`s that can be used during simulation, when actual data from the sensor is not available.

External streams are one example of a stream from which we cannot drop samples, since that would require being able to provide data that has not been produced by the system yet. If we try to drop samples from a stream, for example, by using the expression `drop 1 altitude` anywhere in our specification, Copilot reports a *compile-time* error:

```

1 Copilot error: Drop applied to non-append operation!

```

The error is not reported if we first append samples to the stream, for example, with `drop 1 ([a1, a2] ++ altitude)` (where `a1` and `a2` are two valid and known altitudes). If we drop more samples than we prepend, however, Copilot still reports an error:

```

1 Copilot error: Drop index overflow!

```

Definitions that drop “too much” are disallowed because they present potential violations of causality: the present of a stream may depend on a future that has not happened yet. Problems with non-causal definitions are common in temporal frameworks [1,8]. By introducing the aforementioned checks, Copilot is able to prevent potential causality errors at compile time.

5.2 Structs

Copilot structs are compiled into C structs and made available to the monitor. To be able to generate a correct `struct` definition in C, Copilot structs need to be defined using specific Copilot types. We normally implement structs in Copilot as records made of *fields*, each having a *name* and a *type*. We use the type `Field s t` to represent each field, with `s` being the field name (a type-level literal string), and `t` being the type of the field.

Example 6. To demonstrate how to work with structs, let us use a different example inspired by one of the properties we monitor in our systems: the temperature of a battery in one of the aircraft’s components. The following defines a new Copilot type `Battery` with a field `temperature` of type `Int16`,⁴ which corresponds to a struct in C with a field `temp` of type `int16_t`:

```

1 data Battery = Battery
2 { temperature :: Field "temp" Int16 }

```

⁴ For the sake of simplicity, we omit other fields in the definition of `Battery`.

We provide a limited API to operate on streams of structs or records. Currently, Copilot supports projections, that is, accessing a field of a struct, with the function:⁵⁶

```
1 (#) :: (Typed a, Typed t)
2     => Stream a -> (a -> Field s t) -> Stream t
```

The first argument denotes the stream carrying a struct of type **a**, and the second denotes a field of the struct with name **s** and type **t**. Because structs are first class, they are valid types to be used in streams, and so are their fields.

Example 7. If we have a global C variable **battery** of the struct type generated from the definition of **Battery** above, holding the state of the battery at each step, we can access it from Copilot with the following definition:

```
1 batt :: Stream Battery
2 batt = extern "battery" Nothing
```

We can extract a field of a stream of structs, producing another stream in a type-safe way:

```
1 tempPlus1 :: Stream Int16
2 tempPlus1 = batt # temperature + 1
```

5.3 Arrays

Copilot also includes support for arrays, with an advanced type that includes the length as part of the type of the array. For example, a stream in which each element is an array with 16 elements of type **Int64** would have type **Array 16 Int64**. The presence of the array's length as a type-level natural number serves two purposes: first, it allows the compiler to detect, at compile time, some incorrect accesses (i.e., access out of bounds), and, second, it allows us to generate C without dynamic memory allocation, as all arrays have known, fixed lengths.

Similarly to structs, Copilot provides a limited API to work with **Streams** of **Arrays**. To access specific elements in the array, we provide the operator⁷:

```
1 (.!!) :: (KnownNat n, Typed t)
2       => Stream (Array n t)
3       -> Stream Word32
4       -> Stream t
```

⁵ The signature of **(#)** includes additional constraints. We omit details out of brevity, but this does not change the way that it is used.

⁶ The parentheses around some operator names are a particularity of the host language, but they are not needed when the operators are used in infix position in expressions, as illustrated later in this section.

⁷ The signature of **(.!!)** imposes additional constraints on the array, which we omit, but the arguments and the way they are used are as described.

This operation allows us to access an element of an array, where the position of that element is determined by a number in a stream. The signature of this operator includes a requirement that `n` be a `KnownNat`. That is just a fancy way of saying that it must be a concrete number known at compile time, that is, a specific natural number (as opposed to a variable holding a natural number).

Example 8. We can augment `Battery` with the measurements of the voltages of individual cells by including a new field:

```
1 data Battery = Battery
2   { temperature :: Field "temp"  Int16
3   , voltages    :: Field "volts" (Array 10 Word16)
4   }
```

The field `voltages` is an array of length 10, whose elements have type `Word16`.

Copilot allows us to define streams that access specific elements of these arrays. For example, we can take the `voltages` field from `batt`, extract its first element, and add one to the result.

```
1 volt0Plus1 :: Stream Word16
2 volt0Plus1 = (batt # voltages .!! 0) + 1
```

Copilot is able to detect some incorrect accesses at compile time, if the index within the stream is out of range and the Copilot expression denoting that index is a constant stream. For example, if we had passed 11 as second argument of `(.!!)`, we would have seen a warning during compilation. Nevertheless, it is not generally possible for all streams to detect incorrect accesses in compile time, since the value of the stream containing the index may only be determined at runtime if they depend on some external variable.

5.4 Monitors

The purpose of Copilot is to monitor properties and to raise an alert when an assertion is violated. The Copilot language defines monitors as sequences of *triggers*. A trigger is defined as a stream of Booleans, a C function to be called when the current sample is true, and the arguments to pass to that function:

```
1 trigger :: String -> Stream Bool -> [Arg] -> Spec
```

`Spec` is a type internal to Copilot and represents a specification.

Properties in triggers denote violations, not assertions. Therefore, triggers denote functions to call when samples are **True**, not **False**.

The function to call is given by the first argument as a **String**, and needs to be implemented by the user. Referring to a function that does not exist would lead to a link-time error. If the header files generated by Copilot are included in other C files of the application that uses Copilot, referring to a function with the wrong arguments in a trigger would also lead to a compilation error.

Example 9. The following specification declares a monitor that executes the C function `large`, passing as argument the current value of the stream `counter`, when the voltage of the first cell of the battery, plus one unit, is greater than 8. Arguments are passed as a comma-separated list, with the keyword `arg` preceding each argument stream:

```
1 monitor :: Spec
2 monitor =
3   trigger "large" (volt0Plus1 > 8) [arg counter]
```

Specs represent computations, so we can declare multiple triggers by listing them in sequence, preceded by the language keyword `do` (what is known in the host language Haskell as `do` notation).

We can expand the prior definition by adding a second trigger that calls the function `too_large` with no arguments when the same voltage is greater than 10 (see Sect. 5.3):

```
1 monitor :: Spec
2 monitor = do
3   trigger "large" (volt0Plus1 > 8) [arg counter]
4   trigger "too_large" (volt0Plus1 > 10) []
```

Copilot specifications can be simulated on a computer, or compiled into C code to be used in the same or a different device. In Sect. 7 we demonstrate how the specification can be compiled into C and integrated in a larger system.

6 Logics and Languages

Monitors and specifications can become overly complex as systems grow. To aid understanding, Copilot supports extending the language with new operators without having to modify its internals. Users can leverage such facilities to simplify their specifications and reuse constructs across projects.

This section introduces the temporal and propositional logic libraries of Copilot, which are defined using the aforementioned extension mechanisms. In this tutorial we concentrate on the past-time temporal logics. However, Copilot also includes libraries for future-time linear temporal logics, including bounded LTL and metric temporal logic.

6.1 Logical Operators

As mentioned in Sect. 5, Copilot extends the standard APIs of the supported types to apply pointwise on streams. In the case of Booleans, Copilot provides a number of logical operators based on propositional logic. Apart from the constants `true` and `false`, the following operators on Boolean streams are provided:

```

1 not    :: Stream Bool -> Stream Bool
2 (&&)   :: Stream Bool -> Stream Bool -> Stream Bool
3 (||)   :: Stream Bool -> Stream Bool -> Stream Bool
4 xor    :: Stream Bool -> Stream Bool -> Stream Bool
5 (==>)  :: Stream Bool -> Stream Bool -> Stream Bool

```

In all cases, these operators apply the associated Boolean operation to the values at each sample. For example, given two Boolean streams `s1` and `s2`, the stream `s1 ==> s2` is true at a time (i.e., sample) if `s2` is true at that time, or if `s1` is false.

While these logical operators can help simplify basic expressions, the complexity of real-world applications demands higher-level languages. In the following we explore temporal logics supported by Copilot, and introduce operators to refer to past or future values of a stream.

6.2 Temporal Logics

Generally speaking, temporal logics extend other logics with a temporal dimension. To describe relations between formulas at different times, temporal logic languages introduce modal operators that abstract over time. For example, some languages provide an operator $\Box$ (called *always*, also written G , after the word *globally*), and a formula $\Box \phi$ is true if and only if ϕ is true at all times, where the specific meaning of the expression “at all times” depends on the logic.

Temporal logic languages generally vary in the logic they are based on, the temporal operators they support and in their model of time (e.g., continuous vs discrete, linear vs branching, future and/or past). These aspects impact what formulas can be expressed, which ones are true or false, what information is needed to evaluate them, and how efficiently we can do so.

Because time is an essential component of stream languages, like Copilot, temporal logics constitute a suitable mechanism to express many of the re-occurring patterns in monitor specifications. In the following we discuss some of the languages supported by Copilot, and demonstrate their use with examples.

Past-Time Linear Temporal Logic. Past-time Linear Temporal Logic (ptLTL) is an extension of propositional logic in which time is seen as linear, discrete, and bounded. While, in propositional logic, every variable may take the value true or false, in ptLTL, every variable may take the value true or false, *at each point in the present or in the past*.

Past-time linear temporal logic introduces temporal operators, letting us express logical formulas based not only on certain propositions being true at the present, but also on their validity in the past.

Copilot supports the temporal operators `alwaysBeen`, `eventuallyPrev`, `previous`, and `since`, all operating on and returning Boolean streams:

```

1 alwaysBeen    :: Stream Bool -> Stream Bool
2 eventuallyPrev :: Stream Bool -> Stream Bool

```

```

3 previous      :: Stream Bool -> Stream Bool
4 since         :: Stream Bool -> Stream Bool -> Stream Bool

```

A stream `alwaysBeen x` is true at a time if `x` has been true at all times, present and past (Fig. 4). The operator `eventuallyPrev` works the opposite way, and `eventuallyPrev x` is true if `x` has ever been true. The temporal operator `previous` refers to the immediately previous sample, with `previous x` being true if `x` was true in the last sample. Finally, `since x y` is true at a time if the stream `x` has been continuously true since the first sample after `y` became true.

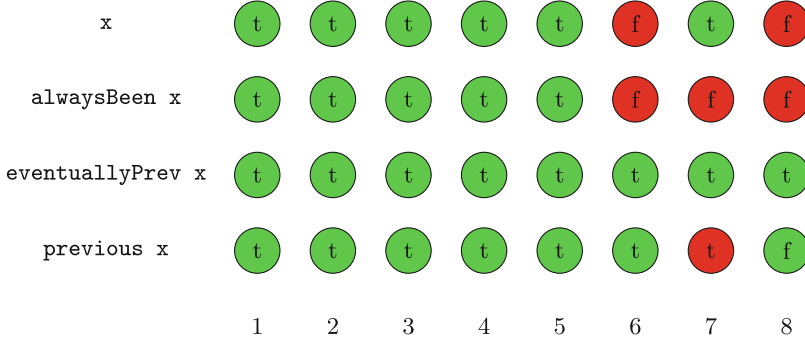


Fig. 4. Example of values of the formulas `alwaysBeen x`, `eventuallyPrev x`, and `previous x`, for different values of `x` at different times.

Example 10. Borrowing the example in the prior section, imagine that we want to detect if the voltage of the first battery cell was ever too high. We can express that in Copilot with the following specification:

```

1 voltageWasTooHigh :: Stream Bool
2 voltageWasTooHigh = eventuallyPrev ((batt # voltages .!! 0) >= 10)

```

We can combine these temporal operators with the pointwise operators presented earlier in this section, to capture, for example, that a safety system can only be activated if a condition was violated before:

```

1 safetyResponseOK :: Stream Bool
2 safetyResponseOK = safetyEngaged ==> voltageWasTooHigh
3 where
4   safetyEngaged :: Stream Bool
5   safetyEngaged = extern "safety_system" Nothing

```

7 Integration Into Larger Systems

The facilities described until now allow users to specify properties to be monitored. To use such monitors in practice, users need to provide the inputs to the monitors and define the functions to handle property violations.

This section describes how to generate the C code for the monitors, and how to connect the monitors to the rest of the system under observation. The purpose is to give the reader a full understanding of the interfaces used by Copilot. However, for larger projects and existing software frameworks, we have developed a separate tool that simplifies the process. Currently, the tool supports NASA's Core Flight System (cFS) [23], FPrime [3], the Robot Operating System 2 (ROS 2), and we welcome contributions to support other platforms.

7.1 Monitor Generation in C

To generate the C code for the monitors, we need to modify the specifications to indicate that we wish to compile the specification to C. Taking the example from Sect. 4 about the altitude / autopilot as a starting point, we first need to add one more import, namely:

```
1 import Copilot.Language.C99
```

We can now add a main function that compiles the specification. We first need to reify it, which in Copilot's compiler is a process that also checks the specification's correctness. The result of the reification process is a post-processes specification that we can compile to C:

```
1 main = do
2   reifiedSpec <- reify monitor
3   compile "rv" reifiedSpec
```

To actually compile the spec and generate the C code, all we need to do is run the specification. Because Copilot is a DSL embedded in Haskell, we use the standard tool `runhaskell` (assuming that the specification is saved in a file called `Spec.hs`):

```
1 $ runhaskell Spec.hs
```

The result of this execution are three files, namely `rv.c`, `rv.h`, and `rv.types.h`. The first file contains the actual implementation of the monitors. We do not need to concern ourselves with how the file is implemented; everything we need to interact with the monitors is declared in `rv.h`. The latter file `rv.types.h` contains local definitions of any auxiliary types used in the specifications (e.g., structs). Since in this case there are none, that file is empty.

Inspecting `rv.h` more closely shows a number of definitions that we need to provide, or that we can use:

```
1 extern bool autopilot;
2 extern uint64_t altitude;
3 void step();
4 extern void recover(bool arg1, uint64_t arg2);
```

The first two declarations are the inputs to the system. It is the Copilot user's obligation to define those somewhere and give them appropriate values at all times and before the monitoring system checks the current state of the

monitors. The next line, `void step();`, declares the main entry point of the monitoring system. The user of this code must call the `step` function when they want to check the status of the monitors. Note that calling `step` also advances the implicit clock for the monitoring system. Finally, the function `recover` must be defined by the user of this code, and must implement a mechanism to recover from the property violation.

7.2 Integration

The following is a sample C code that puts data in both inputs, calls `step`, and prints messages whenever there are violations:

```

1 bool autopilot = false;
2 uint64_t altitude = 0;
3 void recover(bool arg1, uint64_t arg2) {
4     printf("Violation: %d\n", arg2);
5 }
6 int main () {
7     int i = 0;
8     for (i = 0; i < 10000; i++) {
9         autopilot = (i % 1000) < 500;
10        altitude  = abs (i - 500);
11        step();
12    }
13 }
```

In this code, the `recover` function simply reports the violation, but does not actually recover from the off-nominal situation. However, in a real system, the `recover` function could activate a recovery routine that actually brings the plane to a higher altitude. In general, the details of the recovery routine are mission specific and quite complex. If, for example, we were working with a quadcopter instead of a plane, the recovery routine for a loss of altitude might increase the power on the propellers, provided that the quadcopter is level, rightside up, and there are no objects in its path moving upwards, and other necessary safety conditions hold. Other recovery methods for the same situation would be possible, depending on the vehicle and the situation.

8 Conclusion

In this paper, we described Copilot, a runtime verification framework for safety-critical, real-time embedded systems. We discussed the evolution of the project, how it was originally built, and how the components that make the framework have changed over time based on both project need and resources available. The current version of the language was introduced, and we saw how the new constructs of structs and arrays help to deal with more complex data structures without sacrificing safety. Copilot is a project rich in libraries, and we discussed

different temporal logics supported by the language. We also illustrated how to create monitors and integrate them into an existing system with minimal interference with the SUO.

There are a number of open problems in this domain, and in Copilot as a whole, that remain to be addressed. We expect future versions of the language to be simpler and require less boilerplate code, to support generating monitors in languages other than C, and to expose less of the underlying Haskell ecosystem.

Acknowledgements. Figure 3 is based on image “City skyline” by Rg1024 (public domain), and image “Large modern passenger airliner jet”, Designed by macrovector/FreePik.

References

1. Bahr, P., Berthold, J., Elsmann, M.: Certified symbolic management of financial multi-party contracts. In: Proceedings of the 20th ACM SIGPLAN International Conference on Functional Programming, pp. 315–327. ICFP 2015, ACM, New York, NY, USA (2015). <https://doi.org/10.1145/2784731.2784747>
2. Bartocci, E., Falcone, Y., Francalanza, A., Reger, G.: Introduction to runtime verification. In: Bartocci, E., Falcone, Y. (eds.) Lectures on Runtime Verification, pp. 1–33. Springer International Publishing, Cham (2018). https://doi.org/10.1007/978-3-319-75632-5_1
3. Bocchino, R., Canham, T., Watney, G., Reder, L., Levison, J.: F Prime: an open-source framework for small-scale flight software systems (2018)
4. Caspi, P., Paliud, D., Halbwachs, N., Plaice, J.: LUSTRE: a declarative language for programming synchronous systems. In: 14th Symposium on Principles of Programming Languages, pp. 178–188 (1987)
5. Courtney, A., Elliott, C.: Genuinely functional user interfaces. In: Haskell Workshop, pp. 41–69 (2001)
6. D’Angelo, B., et al.: LOLA: runtime monitoring of synchronous systems. In: 12th International Symposium on Temporal Representation and Reasoning, pp. 166–174. IEEE (2005)
7. Dwyer, M., Diep, M., Elbaum, S.: Reducing the cost of path property monitoring through sampling. In: Proceedings of the 23rd International Conference on Automated Software Engineering, pp. 228–237 (2008)
8. Elliott, C., Hudak, P.: Functional reactive animation. In: Proceedings of the Second ACM SIGPLAN International Conference on Functional Programming, pp. 263–273. ICFP ’97, ACM (1997)
9. Goodloe, A.: Challenges in high-assurance runtime verification. In: Leveraging Applications of Formal Methods, Verification and Validation: Foundational Techniques - 7th International Symposium, ISOFA 2016, Imperial, Corfu, Greece, October 10–14, 2016, Proceedings, Part I, pp. 446–460 (2016)
10. Goodloe, A., Pike, L.: Monitoring distributed real-time systems: a survey and future directions. Tech. Rep. NASA/CR-2010-216724, NASA Langley Research Center (2010)
11. Havelund, K., Goldberg, A.: Verify your runs. In: Meyer, B., Woodcock, J. (eds.) Verified Software: Theories, Tools, Experiments, pp. 374–383. Springer, Berlin, Heidelberg (2008). https://doi.org/10.1007/978-3-540-69149-5_40

12. Johannsen, C., Jones, P., Kempa, B., Rozier, K.Y., Zhang, P.: R2U2 Version 3.0: re-imagining a toolchain for specification, resource estimation, and optimized observer generation for runtime verification in hardware and software. In: Enea, C., Lal, A. (eds.) *Computer Aided Verification*, pp. 483–497. Springer Nature Switzerland (2023)
13. Laurent, J., Goodloe, A., Pike, L.: Assuring the guardians. In: Bartocci, E., Majumdar, R. (eds.) *Runtime Verification*, pp. 87–101. Springer International Publishing, Cham (2015). https://doi.org/10.1007/978-3-319-23820-3_6
14. Knight, J.C.: Safety critical systems: challenges and directions. In: *Proceedings of the 24th International Conference on Software Engineering*, pp. 547–550. ICSE '02, ACM (2002)
15. Manna, Z., Pnueli, A.: *The Temporal Logic of Reactive and Concurrent Systems*. Springer, New York, NY (1992). <https://doi.org/10.1007/978-1-4612-0931-7>
16. NASA: NASA Software Engineering Requirements NPR7150.2C. https://nodis3.gsfc.nasa.gov/displayAll.cfm?Internal_ID=N_PR_7150.002C_ (2019)
17. Pike, L., Goodloe, A., Morisset, R., Niller, S.: Copilot: a hard real-time runtime monitor. In: Barringer, H., et al. (eds.) *Runtime Verification: First International Conference, RV 2010, St. Julians, Malta, November 1-4, 2010. Proceedings*, pp. 345–359. Springer, Berlin, Heidelberg (2010). https://doi.org/10.1007/978-3-642-16612-9_26
18. Pike, L., Wegmann, N., Niller, S., Goodloe, A.: Experience report: a do-it-yourself high-assurance compiler. In: *Proceedings of the International Conference on Functional Programming (ICFP)*. ACM (2012)
19. Pike, L., Wegmann, N., Niller, S., Goodloe, A.: Copilot: monitoring embedded systems. *Innov. Syst. Software Eng.* **9**(4) (2013)
20. Pnueli, A.: The temporal logic of programs. In: *Proceedings of the 18th Annual Symposium on Foundations of Computer Science*, pp. 46–57. SFCS '77, IEEE Computer Society, Washington, DC, USA (1977). <https://doi.org/10.1109/SFCS.1977.32>
21. Reger, G., Havelund, K.: What is a trace? A runtime verification perspective. In: Margaria, T., Steffen, B. (eds.) *ISoLA 2016. LNCS*, vol. 9953, pp. 339–355. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-47169-3_25 7th International Symposium on Leveraging Applications of Formal Methods, Verification and Validation (ISoLA 2016); Conference date: 05-10-2016 Through 14-10-2016
22. Scott, R.G., Dodds, M., Perez, I., Goodloe, A.E., Dockins, R.: Trustworthy runtime verification via bisimulation (experience report). *Proc. ACM Program. Lang.* **7**(ICFP) (2023)
23. Wilmot, J.: A core flight software system. In: *Proceedings of the 3rd IEEE/ACM/I-FIP International Conference on Hardware/Software Codesign and System Synthesis*, pp. 13–14. CODES+ISSS '05, ACM, New York, NY, USA (2005). <https://doi.org/10.1145/1084834.1084842>

Open Access This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (<http://creativecommons.org/licenses/by/4.0/>), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

